CACHE PERFORMANCE AND MISS BEHAVIOR

Cache Performance Optimization

Topics:

- Why cache miss
- The 3C model of cache misses
- Conflict behavior
- Replacement policies
- Techniques used in real processors

What is a cache miss?

A cheche miss occurs when requested data is not present in cache

Miss consequences:

1. Access lower level memory
2. Fetch block
3. Replace existing block

Cost = miss penalty

In modern system this can exceed 200 cycles

Why do misses occur?

Misses occur for three fundamental reasons

They were formalized in the 3C model:

1. Compulsory misses
2. Capacity misses
3. Conflict misses

Understanding these helps guide architectural design

Compulsory misses

Also called cold misses

Occur when data is accessed for the first time

Characteristics:

- Cannot be avoided by cache size
- Only reduced by prefetching

Example: first iteration of a loop accessing an array

Capacity misses

Occurs when the working set exceeds cache size

Even with perfect mapping, cache cannot hold all blocks

Solution:

- Larger caches
- Multi-level caches

Example: cache is 64 KB but the program uses 2 MB of active data

Conflict misses

Occurs when multiple blocks map to same cache set

Common in direct-mapped caches and low associative designs

Solution:

- Increase associativity
- Better indexing schemes

Example: two arrays mapping to same index: they repeatedly evict each other

Cache misses - example

A processor has 32 bit addresses and a 256 lines direct-mapped cache, with a 16 B block. Compute the total misses of the following assembly code.

```
        la $t0, 0xC0ABD030
        la $t1, 0xD8EFC030
        li $t2,32
        li $t3, 0
loop:   lbu $t4,0($t0)
        lbu $t5,0($t1)
        addiu $t0, $t0, 1
        addiu $t1, $t1, 1
        addu $t3, $t4, $t1
        addiu $t3, $t3, 1
        bne $t3, $t2, loop
```

Cache misses - example

```
            la $t0, 0xC0ABD030
            la $t1, 0xD8EFC030
            li $t2,32
            li $t3, 0
loop:  lbu $t4,0($t0)
            lbu $t5,0($t1)
            addiu $t0, $t0, 1
            addiu $t1, $t1, 1
            addu $t3, $t4, $t1
            addiu $t3, $t3, 1
            bne $t3, $t2, loop
```

Memory content:

0xC0ABD030 -> 32 B vector A

0xD8EFC030 -> 32 B vector B

Address decomposition:

16 B block -> 4 bit offset

256 lines -> 8 bit cache index

32 – 4 – 8 = 20 bit tag

Cache misses - example

	validity bit	tag	data
0	0		
1	0		
2	0		
3	0		
4	0		
	.		
	.		
	.		
254	0		
255	0		

```
        la $t0, 0xC0ABD030
        la $t1, 0xD8EFC030
        li $t2,32
        li $t3, 0
loop:   lbu $t4,0($t0)
        lbu $t5,0($t1)
        addiu $t0, $t0, 1
        addiu $t1, $t1, 1
        addu $t3, $t4, $t1
        addiu $t3, $t3, 1
        bne $t3, $t2, loop
```

Cache misses - example

First load: lbu $t4,0($t0)

$t0 contains the address 0xC0ABD030 -> offset =0

cache index = 03

tag = C0ABD

Compulsory miss and caching of the first 16 B of vector A

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	C0ABD	16 B of vector A
4	0		
.			
.			
254	0		
255	0		

Cache misses - example

Second load: lbu $t5,0($t1)

$t1 contains the address 0xD8EFC030 -> offset =0

cache index = 03

tag = D8EFC

Conflict miss and caching of the first 16 B of vector B

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	D8EFC	16 B of vector B
4	0		
.			
.			
254	0		
255	0		

Cache misses - example

The loop iterates over 32 iterations

At each iteration there are 2 misses

These two misses are conflict misses (only in the first and sixteenth iteration one is compulsory misses)

The total number of misses is then 64!

More in detail 63 conflict misses and 1 compulsory misses

Access pattern A B A B A B -> miss rate = 100%

Cache misses - example

Higher associativity reduces conflict misses

In this example, if we assume a 2-way cache we have 2 blocks per set

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	C0ABD	16 B of vector A
4	0		
.			
.			
254	0		
255	0		

	validity bit	tag	data
0	0		
1	0		
2	0		
3	1	D8EFC	16 B of vector B
4	0		
.			
.			
254	0		
255	0		

Cache misses - example

In the first iteration we have 2 compulsory misses

At the 16^{th} iteration we have 2 compulsory misses (block size is 16 B, array A and B are 32 B each)

No conflict misses!

Miss rate = 4/64 = 6.25 %

Replacement Problem

When cache is full, we must choose a block to evict

This is the replacement policy

Goal: evict block least likely to be used

Optimal policy: evict block whose next use is farthest in the future

Impossible in practice

It provides upper bound on performance

Last Recently Used (LRU) Replacement

Block not used recently unlikely to be used soon

Works well when temporal locality is strong

Used historically in many caches

LRU requires tracking usage order

For N-way caches LRU state complexity grows

Pseudo LRU

Approximates LRU with lower cost

Trade-off:

- Slightly worse miss rate
- Much lower hardware cost

Popular techniques:

- Tree-based PLRU
- Not Recently Used (NRU)

Random Replacement

Simple strategy: choose a random block

Surprisingly effective in large caches

Advantages:

- Very low hardware complexity
- Avoids pathological patterns

Used in some GPUs

The write problem

So far we focused on reads

Writes introduce additional complexity

When the CPU writes to a cache block, what should happen?

Two fundamentals write policies:

- Write-Through (WT) -> write goes to cache and to memory immediately
- Write-Back (WB) -> write goes only to cache and the memory is updated later, on eviction

Write-Through

Characteristics:

- ✅ Simple
- ✅ Memory always consistent
- ✅ Easy for I/O coherence

- ❌ High memory traffic
- ❌ Lower performance

Often used in L1 caches (historically) and simple systems

Write-Back

Characteristics:

- ✅ Lower memory traffic
- ✅ Better performance
- ✅ Exploits temporal locality

- ❌ More complex
- ❌ Requires dirty bit
- ❌ More complex coherence handling

Used in most modern CPUs (L2/L3 always WB)

Write-Back – Dirty bit concept

Each cache block has:

- Validity bit
- Tag
- Dirty bit

Dirty = modified in cache but not in memory

On eviction:

if dirty -> write to memory

if clean -> discard block

Write Miss Policies

What happens on a write miss?

Two options:

Write Allocate (fetch on write)

- Load block into cache and then write

No Write Allocate (Write-around)

- Write directly on memory and do not load block into cache

Policy combination

Common combinations

Policy	**Miss Policy**
Write-through	No Write Allocate
Write-back	Write Allocate

Why?

Write-back benefits form locality -> keep data in cache
Write-through avoids cache pollution

Impact on Performance

Write policies affect:

- Memory bandwidth usage
- Energy consumption
- Latency
- Coherence traffic

In multicore systems:

Write-back Increases coherence complexity
Write-through increases interconnect traffic

Real System Design Choices

Typical modern CPUs:

- L1:
  - Often write-back
  - Sometimes write-through (older/simpler designs)

- L2/L3:
  - Always write-back

Reason:

Reduce memory bandwidth pressure and improve energy efficiency

Engineering Trade-Off Summary

Aspect	**Write-Through**	**Write-Back**
Memory traffic	High	Low
Complexity	Low	High
Performance	Lower	Higher
Coherence	Easier	Harder

Techniques to Reduce Misses

There are several strategies

1. Increase cache size
   Reduce capacity misses
2. Increase associativity
   Reduce conflict misses
3. Larger blocks
   Exploits spatial locality
4. Hardware prefetching
   Reduce compulsory misses

Hardware Prefetching

Prefetching anticipates future access

Benefits:

- Hides memory latency
- Reduce compulsory misses

Risk:

- Useless prefetches increase bandwidth pressure

Example: sequential array traversal. Prefetch next block before it is requested.

Engineering Perspective

Cache performance depends on:

- Workload access pattern
- Data structure layout
- Compiler optimizations
- Hardware prefetching
- Cache hierarchy

Memory performance is software-hardware co-design!

Closing

Summary of key concepts:

- Why caches miss
- The 3 C miss model
- Replacement policies
- Techniques to reduce misses

Next episode: multi-level cache hierarchies (L1/L2/L3 interactions, inclusive vs exclusive caches and modern CPU design)